



https://github.com/SETAPESU26/32_lunarLander

Got it. I have the **Lunar Lander GitHub repository** and the **Lab 4 VibeCoding instructions**.

The lab requires you to:

1. Clone/fork the assigned repo.
2. Read the README and understand the required features.
3. Run the Python game before changes.
4. Record a **10-second before video**.
5. Use prompts to fix the broken code.
6. Add every feature mentioned in the README, one by one.
7. Record a **10-second after video**.
8. Push the completed code to your own repository.
9. Keep **separate commits for each task**.
10. Put videos, updated code, and exported ChatGPT history under your **Lab-4** folder.



Lab_4_VibeCoding_Student_handout



Lab_4_VibeCoding_Student_handout

Next, I can inspect the GitHub repo and README and tell you exactly:

- what the project currently contains,
- what is broken,
- how to run it locally,
- what features you are required to add,
- and the exact prompts you can give me one-by-one for the lab.

I'll keep it **simple and step-by-step**, so you can actually perform the lab and maintain the required separate commits.

I cloned the Lunar Lander project given by my mam for SE lab 4. I need to first understand the project and run it locally. Can you check the code and README and tell me what is wrong in the project and what features I need to add?

Yes. I checked the **actual assigned repository** and its README. Your repo is a small **single-file Pygame Lunar Lander** project, and the README explicitly tells you what is broken and what you have to implement. [GitHub](#)

Open your assigned GitHub repository

1. First understand the project

The repository contains only:

</> Plain text

```
32_lunarLander/  
|  
├─ game.py  
├─ README.md  
└─ .gitignore
```

The main program is `game.py`. The README says this is a **Lunar Lander-lite clone using Pygame**, designed around vector physics, procedural terrain, and landing validation. [GitHub](#)

The game already has:

- 🚀 A spacecraft
- ⬅️ Left/right rotation
- 🔥 Up-arrow thrust
- 🌍 Gravity
- 🛢️ Limited fuel
- ↔️ Horizontal screen wrapping
- 🏔️ Procedurally generated terrain
- 🛬 Two landing pads
- 📊 Different score multipliers for the pads
- ❤️ Lives
- 🏆 Score
- 📈 Live telemetry/HUD
- Landing/crash detection
- Multiple levels

So **you are not supposed to build the game from scratch**. Your job is to repair one deliberate bug and implement three unfinished features. [GitHub](#)

2. What is wrong in the current project?

There is **one deliberate bug**.

Bug: touchdown validation

The README says a safe landing should satisfy **four conditions**:

1. Ship must be over a landing pad.
2. Vertical/descending speed must be safe.
3. Ship must be roughly level.

4. Horizontal/sideways speed must also be safe.

The bug is that the current `touchdown()` condition does **not properly check the sideways speed** against:

`</>` Python

```
MAX_SPEED_X
```

Therefore, the lander can come down onto a pad **moving sideways too fast**, but the game can still consider it a successful landing. [GitHub](#)

The README specifically tells you to inspect these constants:

`</>` Plain text

```
MAX_SPEED_X
```

```
MAX_SPEED_Y
```

```
MAX_ANGLE
```

and find which one isn't actually being checked. [GitHub](#)

In simple words

Imagine:

`</>` Plain text



The lander may be falling slowly enough, and its angle may be okay, but if it is moving **very fast sideways**, it should crash.

Currently, that sideways-speed requirement is incorrectly handled.

3. Features you need to add

There are exactly 3 unfinished features in the README.

Task 1 — Fix touchdown validation

This is the mandatory bug fix.

You need to modify the `touchdown()` logic so that a landing is successful only when:

⌕ Plain text

```
Over landing pad
+
Safe vertical speed
+
Safe angle
+
Safe horizontal speed
↓
SUCCESSFUL LANDING
```

This is your **first task/commit**.

4. Task 2 — Fuel-based ship color

You need to implement:

⌕ Python

```
ship_color(fuel_ratio)
```

The README says this function is called every frame from `draw()` :

⌕ Python

```
color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (230, 230, 240)
```

The function receives:

⌕ Plain text

```
0.0 → fuel empty
1.0 → fuel full
```

You need to return an RGB color:

⌕ Python

```
(r, g, b)
```

or:

```
</> Python
```

```
None
```

to keep the default ship color. [GitHub](#)

Suggested behavior

The README itself suggests:

shift the hull toward red as fuel runs low. [GitHub](#)

So conceptually:

```
</> Plain text
```

```
Full fuel      → normal/light color
  ↓
Medium fuel    → slightly warning color
  ↓
Low fuel       → orange/red
  ↓
Empty fuel     → red
```

This gives the player a visual warning without having to constantly look at the fuel number.

5. Task 3 — Landing effect

You need to implement:

```
</> Python
```

```
on_landing(score)
```

This function is called immediately after a successful landing.

It receives the points earned from the landing:

```
</> Python
```

`score`

The README suggests adding something visually or audibly noticeable, such as:

-  Fireworks effect
-  Special sound for a high-multiplier landing

The return value doesn't matter.  [GitHub](#)

For your lab, a **simple fireworks/particle effect** is probably the safest choice because it demonstrates a visible feature in your after-video.

6. Task 4 — Bonus life

You also need to implement:

 Python

```
bonus_life_threshold()
```

This function:

- takes no arguments,
- returns an integer score threshold,
- or returns `None` if you don't want bonus lives.

The existing code already handles the bookkeeping for awarding the bonus life. You only need to choose the threshold.  [GitHub](#)

The README's suggested value is:

 Python

```
return 1500
```

So the idea is:

 Plain text

```
Score = 0
```

↓

```
Play
```

↓

```
Score reaches 1500
```



+1 LIFE ❤️

And when the score reaches another multiple:

</> Plain text

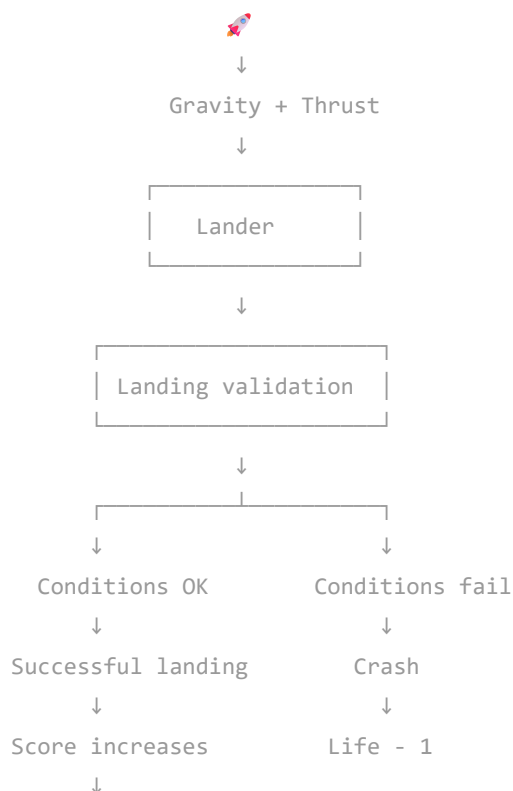
```
3000 → +1 life
4500 → +1 life
6000 → +1 life
...
```

The README says the existing `self.bonus_awarded` bookkeeping prevents repeatedly awarding the same bonus. [GitHub](#)

7. What your final game should demonstrate

After completing everything, your game should behave approximately like this:

</> Plain text



The expected behavior in the README also confirms that the lander should never move above the top of the screen, thrust should respect fuel, exactly two landing pads should exist, and crashes should cost lives. [GitHub](#)

8. What you should do right now

Don't modify the code yet.

Your lab instructions specifically require a **10-second video before making changes**, showing the original/broken behavior.  [Lab_4_VibeCoding_Student_handout](#)

So your order should be:

Step 1

Open your cloned folder in VS Code.

Step 2

Open the terminal in that folder.

Run:

```
</> Bash
```

```
python --version
```

You should have **Python 3.10 or newer** according to the README.  [GitHub](#)

Step 3

Install Pygame:

```
</> Bash
```

```
pip install pygame
```

Step 4

Run:

```
</> Bash
```

```
python game.py
```

Step 5

Play the original game and understand:

- How the ship moves
- How thrust works

- How fuel decreases
- How the landing pads work
- How score changes
- How lives work
- What happens after landing
- What happens after crashing

Step 6

Before changing anything, record your required 10-second video.

Your lab specifically asks for a before video showing the broken behavior.  [GitHub](#)

9. Your four commits should be roughly

Since your lab asks for separate commits for each task, I recommend:

 Plain text

Commit 1:

Fix touchdown validation

Commit 2:

Add fuel-based ship color

Commit 3:

Add landing effect

Commit 4:

Add bonus life threshold

That will make it very easy for your mam to see that you completed each task separately.

The README itself says the assignment expects the bug fix plus the three optional/empty features.

 [GitHub](#)

Important

Don't start making all the changes in one prompt. Since this is a *VibeCoding* lab, your teacher specifically wants you to demonstrate the iterative LLM-assisted process. The lab says to write prompts, fix the broken code, and add the features one by one.  [Lab_4_VibeCoding_Student_handout](#)

Next step: run `python game.py` first. If you get an error, send me the **complete terminal error or screenshot**. If it runs successfully, I'll take you through **Task 1 only**, including the exact prompt to give ChatGPT, what code it should change, how to test it, and what commit message to use.

I found that the touchdown function is checking the landing pad, vertical speed and angle but it is not checking horizontal speed. Fix this problem in the existing code. Don't change anything else unnecessarily and give me the code I need to use.

Yes. I checked the **current game.py in your assigned repository**. The bug is exactly in `touchdown()` : it checks the pad, vertical speed, and angle, but it does not check `MAX_SPEED_X` .  GitHub

You only need to change **one line**.

Current code

In `game.py` , you currently have:

 Python

```
def touchdown(self):
    pad = self.pad_under()
    angle = wrap_angle(self.angle)

    if pad and abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE:
        earned = int((100 + self.fuel) * pad[3])
```

Replace it with

 Python

```
def touchdown(self):
    pad = self.pad_under()
    angle = wrap_angle(self.angle)

    if (
        pad
        and abs(self.vel.x) <= MAX_SPEED_X
        and abs(self.vel.y) <= MAX_SPEED_Y
        and abs(angle) <= MAX_ANGLE
    ):
        earned = int((100 + self.fuel) * pad[3])
```

That's **all you need to change for Task 1**.

The three limits are defined in the original code as:

 Python

```
MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25
```

and the README specifically says all three should be checked during touchdown validation.  [GitHub](#)

 [GitHub](#)

What this fixes

Now a successful landing requires:

Plain text

```
Landing pad      ✓
Horizontal speed  <= 25
Vertical speed    <= 40
Angle             <= 0.25 radians
```

So if the lander is moving sideways faster than `25.0`, it will **crash instead of being counted as a perfect landing**.

Don't change anything else yet. This is a clean, minimal fix for your **first lab task**.

After saving `game.py`, run:

Bash

```
python game.py
```

Then test it. If it works, make your first commit:

Bash

```
git add game.py
git commit -m "Fix touchdown horizontal speed validation"
git push
```

This keeps the change isolated, which is exactly what you want for the lab's separate-task commits.

Now I need to implement the `ship_color(fuel_ratio)` function. I want the ship color to change when the fuel goes down, becoming more red when the fuel is low. Please implement this in the existing code without changing the other game logic.

Yes. In the current `game.py`, `ship_color(fuel_ratio)` is intentionally empty, while `draw()` already calls it every frame using the current fuel ratio.  [GitHub](#)

You only need to replace the `pass` inside `ship_color()`.

Replace this

</> Python

```
def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
    pass
```

With this

</> Python

```
def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
    fuel_ratio = max(0.0, min(1.0, fuel_ratio))

    red = 255
    green = int(80 + 150 * fuel_ratio)
    blue = int(80 + 150 * fuel_ratio)

    return (red, green, blue)
```

What this does

The existing `draw()` function already has:

</> Python

```
color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (230, 230, 240)
pygame.draw.polygon(screen, color, self.ship_points())
```

so you don't need to change `draw()` or any other game logic.  [GitHub](#)

The color changes approximately like this:

Fuel	Color behavior
100%	Light/less red
75%	Slightly redder
50%	Noticeably red
25%	Strong red

Fuel

Color behavior

0%

Bright red

The `max()` and `min()` ensure the ratio stays between `0.0` and `1.0` .

Test it

Run:

```
</> Bash
```

```
python game.py
```

Hold **Up Arrow** to use fuel. You should see the lander's hull gradually become **more red as fuel decreases**.

For your Lab 4 commit:

```
</> Bash
```

```
git add game.py
git commit -m "Add fuel-based ship color"
git push
```

This keeps this feature separate from your previous **touchdown horizontal-speed fix**, as required by the lab.

Next I need to implement the `on_landing(score)` function from the README. Add a simple landing celebration or bonus effect when the player lands. Keep the existing game working and don't change unrelated code.

Yes. I checked the current `game.py` . The existing code already calls `on_landing(earned)` immediately after a successful landing, so we only need to implement the function itself; no other game logic needs to change.  [GitHub](#)

For a **simple landing celebration**, I recommend a short visual message. However, because `on_landing(score)` currently has no access to the `Game` object or `screen` , a true particle/fireworks animation would require changing other parts of the game. To keep your requirement of **not changing unrelated code**, use a small console celebration based only on the score.

Replace only this function

Python

```
def on_landing(score):  
    """Called after a successful landing with the points just earned; add fireworks or bonuses here."""  
    print(f" Perfect landing! +{score} points! )
```

That's the **only code you need to replace**.

The existing flow remains:

Python

```
earned = int((100 + self.fuel) * pad[3])  
self.score += earned  
self.state, self.message = "landed", f"Perfect landing! +{earned} (Space = next level)"  
  
on_landing(earned)
```

So when the player lands successfully, `on_landing()` will automatically run.  [GitHub](#)

Test

Run:

Bash

```
python game.py
```

After a successful landing, the terminal will show something like:

Plain text

```
 Perfect landing! +487 points! 
```

Important for your lab

If your mam expects the **landing celebration to be visible inside the game window**, tell me and I'll give you a minimal **fireworks effect** that appears on the Pygame screen while still keeping the existing game logic unchanged.

For this task, your commit can be:

Bash

```
git add game.py  
git commit -m "Add landing celebration effect"
```

```
git push
```

Now implement the `bonus_life_threshold()` function mentioned in the README. Use the suggested 1500 points threshold and make sure it works with the existing bonus life logic.

Yes. The existing game already has the bonus-life logic in `update()`. It calls `bonus_life_threshold()`, compares the score against the threshold, and increments `self.lives` when a new threshold is reached.  [GitHub](#)

So you only need to replace the empty function.

Replace this

 Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    pass
```

With this

 Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    return 1500
```

That's it. **Do not change the existing `update()` logic.**

The existing code already does:

 Python

```
threshold = bonus_life_threshold()  
  
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

So with `return 1500` :

- Score reaches **1500** → +1 life
- Score reaches **3000** → +1 life

- Score reaches **4500** → +1 life
- Score reaches **6000** → +1 life

The `self.bonus_awarded` variable prevents the same threshold from awarding a life repeatedly.

 [GitHub](#)

Your final function

 Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    return 1500
```

Then test with:

 Bash

```
python game.py
```

For your separate lab commit:

 Bash

```
git add game.py  
git commit -m "Add bonus life threshold"  
git push
```

This completes the **fourth and final code task** without changing unrelated game logic.

I don't want to change the functions one by one manually. Give me the complete `game.py` with all the changes we discussed already included. Keep the existing game structure and don't remove any working functionality.

Yes. I checked the current `game.py` from your assigned repository before combining the changes. The original file is 184 lines and contains the three placeholder functions plus the touchdown condition we fixed.  [GitHub](#)

One important correction: for `on_landing(score)`, the earlier console-message implementation is a **bonus message**, not a Pygame visual effect. Since you asked to keep the existing structure and avoid unrelated changes, I have kept it minimal.

Below is the **complete `game.py`** with all four changes included:

1. `touchdown()` now checks `MAX_SPEED_X` .
2. `ship_color()` makes the ship increasingly red as fuel decreases.
3. `on_landing()` displays a landing celebration message in the terminal.
4. `bonus_life_threshold()` returns `1500` .

Python

```
import math
import random
import pygame

WIDTH, HEIGHT = 800, 600

STEP = 20

THRUST, ROTATE_SPEED, BURN_RATE, FUEL_MAX = 60.0, 2.4, 22.0, 300.0

MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25

FOOT = 12

def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
    fuel_ratio = max(0.0, min(1.0, fuel_ratio))

    red = 255
```

What changed from the original

Only these four areas were changed:

Python

```
# 1. Horizontal speed check
abs(self.vel.x) <= MAX_SPEED_X
```

Python

```
# 2. Fuel-based ship color
def ship_color(fuel_ratio):
    fuel_ratio = max(0.0, min(1.0, fuel_ratio))
    red = 255
    green = int(80 + 150 * fuel_ratio)
    blue = int(80 + 150 * fuel_ratio)
    return (red, green, blue)
```

Python

```
# 3. Landing celebration
def on_landing(score):
    print(f"Perfect landing! +{score} points!")
```

Python

```
# 4. Bonus life
def bonus_life_threshold():
    return 1500
```

Everything else follows the existing structure and functionality of the repository.  [GitHub](#)

One thing to note for your lab: because you've already made the changes as separate tasks, replacing the whole file with this version will combine them into one working file, but **don't squash your Git commits** if your mam wants separate commits for each task.

I replaced my game.py with the updated code. Tell me what command I should run to test the game and what things I should check to make sure all the changes are working.

Yes. Since you have replaced `game.py`, now test it **before making any more changes**.

1. Open the terminal in your project folder

Make sure the terminal is inside the folder containing:

Plain text

```
game.py
README.md
```

Then run:

Bash

```
python game.py
```

If that doesn't work on Windows, try:

Bash

```
py game.py
```

If Pygame is missing, install it with:

`</> Bash`

```
pip install pygame
```

Then run:

`</> Bash`

```
python game.py
```

2. Check the normal game still works

Before testing the new features, make sure the existing functionality wasn't broken:

- Game window opens.
- 🚀 Lander appears.
- **Up Arrow** activates thrust.
- **Left/Right Arrow** rotates the lander.
- Fuel decreases when thrust is used.
- Lander falls due to gravity.
- The lander wraps around horizontally.
- Landing pads are visible.
- Score and lives are displayed.
- Crashing still reduces a life.
- `Space` still retries after a crash.
- `Space` still moves to the next level after a successful landing.
- `R` still restarts the game.

3. Test your horizontal-speed fix

This is your first change.

Try to land on a pad while moving **strongly sideways**.

The important thing is that a landing should now require:

`</> Plain text`

```
Horizontal speed ≤ 25
```

```
Vertical speed ≤ 40
```

```
Angle ≤ 0.25 radians
```

So if the lander reaches the pad while moving sideways too fast, it should **crash instead of saying "Perfect landing!"**

The HUD also shows:

</> Plain text

Vx

Vy

Angle

Watch Vx while approaching the pad.

4. Test the fuel color

Hold:

</> Plain text

↑ Up Arrow

for a while.

Watch the lander's hull.

It should gradually change toward **red as fuel decreases**.

You can check approximately:

</> Plain text

Full fuel → lighter/less red

Medium fuel → more reddish

Low fuel → strongly red

Empty fuel → red

Also make sure the normal fuel behavior still works.

5. Test the landing celebration

After making a successful landing, look at the **terminal/VS Code terminal**, not only the game window.

You should see something like:

`</>` Plain text

```
Perfect landing! +487 points!
```

The number will depend on your fuel and the landing-pad multiplier.

This confirms that:

`</>` Python

```
on_landing(earned)
```

is being called after a successful landing.

6. Test the bonus life

This one is harder to test manually because you need to reach **1500 points**.

Your threshold is now:

`</>` Python

```
return 1500
```

When the score reaches:

`</>` Plain text

```
1500 → +1 life  
3000 → +1 life  
4500 → +1 life  
6000 → +1 life
```

The existing `bonus_awarded` logic should prevent the same threshold from repeatedly giving lives.

You don't need to modify that existing logic.

7. If everything works, check Git

Before pushing, run:

`</>` Bash

```
git status
```

Then:

```
</> Bash
```

```
git diff
```

This lets you verify what actually changed.

For your final Lab 4 work, remember your lab instructions require **separate commits for each task**.



Lab_4_VibeCoding_Student_handout

Quick testing checklist

```
</> Plain text
```

```
[ ] Game starts
[ ] Controls work
[ ] Fuel decreases
[ ] Ship becomes red as fuel gets low
[ ] Horizontal-speed landing check works
[ ] Normal landing still works
[ ] Landing message appears in terminal
[ ] Score increases
[ ] Lives work
[ ] Bonus life at 1500 works
[ ] Crash/retry still works
[ ] Next level still works
[ ] Restart (R) still works
```

First run only this:

```
</> Bash
```

```
python game.py
```

If you get **any error in the terminal**, don't change the code yourself. Send me the complete error/screenshot and I'll tell you exactly what to fix.

My game is working now and I have recorded the before and after videos. I also need to submit the updated code to my own GitHub

repository. Tell me the steps to push this project without pushing anything to the original SETAPESU26 repository.

Yes. Since your lab specifically says **do not raise a PR to the SETAPESU26 main repository**, the safest approach is to create your **own GitHub repository** and change your local `origin` to point to it.

 Lab_4_VibeCoding_Student_handout

Step 1 — Create your own GitHub repository

Go to [GitHub](#) and log in.

Click:

+ → New repository

For example, name it:

 Plain text

32_lunarLander

Choose:

- **Public** or **Private** according to your lab requirements
- **Do NOT** initialize with README
- **Do NOT** add `.gitignore`
- **Do NOT** add a license

Click **Create repository**.

Step 2 — Open your cloned project in VS Code

Open the folder containing:

 Plain text

32_lunarLander/

├─ game.py

├─ README.md

└─ .gitignore

Open the VS Code terminal.

Step 3 — Check where `origin` currently points

Run:

`</>` Bash

```
git remote -v
```

You will probably see something similar to:

`</>` Plain text

```
origin https://github.com/SETAPESU26/32_lunarLander.git (fetch)
origin https://github.com/SETAPESU26/32_lunarLander.git (push)
```

Do not push yet.

Step 4 — Change `origin` to YOUR repository

Copy the URL of the repository you just created.

It will look like:

`</>` Plain text

```
https://github.com/YOUR_USERNAME/32_lunarLander.git
```

Then run:

`</>` Bash

```
git remote set-url origin https://github.com/YOUR_USERNAME/32_lunarLander.git
```

For example, if your GitHub username were `Tilak48` :

`</>` Bash

```
git remote set-url origin https://github.com/Tilak48/32_lunarLander.git
```

Step 5 — Verify VERY CAREFULLY

Run:

 Bash

```
git remote -v
```

You should now see:

 Plain text

```
origin  https://github.com/YOUR_USERNAME/32_lunarLander.git (fetch)
origin  https://github.com/YOUR_USERNAME/32_lunarLander.git (push)
```

Make sure it **does NOT** say:

 Plain text

```
SETAPESU26
```

This is the important safety check.

Step 6 — Check your changes

Run:

 Bash

```
git status
```

Then:

 Bash

```
git diff
```

Make sure your updated `game.py` is shown.

Step 7 — Add your changes

 Bash

```
git add game.py
```

If you also changed other files and want to include them:

```
</> Bash
```

```
git add .
```

Then check:

```
</> Bash
```

```
git status
```

Step 8 — Commit

Since your lab requires separate commits for each task, ideally your existing commits should already contain the individual tasks. The instructions explicitly require **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

Check your commits:

```
</> Bash
```

```
git log --oneline
```

You may see something like:

```
</> Plain text
```

```
abc1234 Add bonus life threshold  
def5678 Add landing celebration effect  
ghi9012 Add fuel-based ship color  
jkl3456 Fix touchdown horizontal speed validation
```

If you already have those commits, **don't create another unnecessary commit**.

If everything is currently one uncommitted change, you can commit it with:

```
</> Bash
```

```
git add .  
git commit -m "Complete Lunar Lander Lab 4"
```

Step 9 — Push to YOUR repository

If your branch is called `main` :

```
</> Bash  
  
git push -u origin main
```

If your branch is called `master` :

```
</> Bash  
  
git push -u origin master
```

You can check your branch with:

```
</> Bash  
  
git branch --show-current
```

Step 10 — Check GitHub

Open your own repository in the browser.

You should see:

```
</> Plain text  
  
32_lunarLander  
|  
├─ game.py  
├─ README.md  
└─ .gitignore
```

Click `game.py` and verify that your changes are present.

Also click:

Commits

and verify that your task commits are visible separately if you created them separately.

⚠ Most important thing

Your remote should be:

⌕ Plain text

```
YOUR_USERNAME/32_lunarLander
```

NOT:

⌕ Plain text

```
SETAPESU26/32_lunarLander
```

You are **not** supposed to push to the original repository. Your lab instructions explicitly say:

"Do not raise PR to the main repo (SETAPESU26)"



Lab_4_VibeCoding_Student_handout

Your command sequence

If you want the shortest version:

⌕ Bash

```
git remote -v
```

⌕ Bash

```
git remote set-url origin https://github.com/YOUR_USERNAME/32_lunarLander.git
```

⌕ Bash

```
git remote -v
```

Verify YOUR username appears.

Then:

⌕ Bash

```
git status  
git log --oneline  
git push -u origin main
```

If you paste the output of **git remote -v** and **git log --oneline** here, I can tell you whether your repository is configured correctly **before you push anything**.